Ejercicios Extra de Análisis de Algoritmos

Jaime C Smith

08/05/2026

1. Sum of first n natural numbers (two versions)

Version 1 – manual_add

```
def manual_add(n):
    result = 0
    for i in range(1, n + 1):
        result += i
    return result
```

Time complexity:

- The for loop runs from 1 to n, so it performs n iterations.
- Each iteration does constant work (one addition).
- Overall time complexity: O(n) (linear in n).

Version 2 – add_formula

```
def add_formula(n):
    return n * (n + 1) // 2
```

Time complexity:

- This version does a fixed number of operations: one multiplication, one addition, and one integer division.
- The number of operations does **not** depend on n.
- Overall time complexity: **O(1)** (constant time).

Which version to use for n = 1 000 000 000?

- For n = 1,000,000,000, **Version 2 (add_formula) is clearly better**.
- Reason:
  - manual_add would need to loop **1 billion times**, which is slow and may take noticeable time to finish (linear work).
  - add_formula computes the result in constant time with just a few arithmetic operations, regardless of how large n is.
- So: **Use add_formula because it is O(1) instead of O(n).**

2. linear_search vs binary_search

```
def linear_search(my_list, target):
    for item in my_list:
        if item == target:
            return True
    return False
```

Time complexity (worst case): O(n)

- Let n be the length of my_list.
- In the worst case (target not found or located at the last position), the loop checks all n elements.
- One pass over the list → linear time.

binary_search

```
def binary_search(my_list, target):
    low = 0
    high = len(my_list) - 1

    while low <= high:
        mid = (low + high) // 2
        if my_list[mid] == target:
            return True
        elif my_list[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    return False
```

Time complexity (worst case): O(log n)

- Each iteration of the while loop cuts the remaining search range in **half** (either low moves up or high moves down).
- If the list length is n, you can only halve it $\log_2 n$ times before there is nothing left.
- That gives logarithmic time, O(log n).

When to use each algorithm?

- **Use linear_search when:**
    - The list is **small**, or
    - The list is **not sorted**, and you don't want to sort it, or

- You will only search **once or a few times**, so sorting first is not worth the cost.
- It works on **any** list, no assumptions required.

- **Use binary_search when:**
  - The list is **sorted** (this is required), and
  - You plan to search **many times** in the same list, and/or
  - The list is **large**, so O(log n) is much faster than O(n).
  - Example: searching many times in a sorted user ID list or index.

What if the list is not sorted?

- Binary search **does not work** correctly if the list is not sorted.
- If my_list is unsorted and you still call binary_search, it may:
  - Return False even if the element exists, or
  - Return True for the wrong position (undefined behavior).
- In an **unsorted** list, you have two realistic options:

1. Use linear_search directly: O(n) per search.

2. Sort the list once (O(n log n)), then use binary_search (O(log n) per search) if you will perform many searches and the sorting cost is worth it.

3. print_all_pairs on a dictionary

```
def print_all_pairs(my_dict):
    for key1 in my_dict:
        for key2 in my_dict:
            print(f"{key1}-{key2}")
```

Time complexity

- Let n be the number of keys in my_dict.
- There are **two nested loops**, both iterating over all keys:
  - Outer loop over all n keys.
  - For each key1, inner loop over all n keys again.
- Total number of print operations $\approx n \times n = n^2$.
- Each print is constant time, so total work is proportional to n^2.

- **Time complexity (worst case): O(n²)**

How long if there are 1 million keys?

- If there are **1,000,000 keys** (n = 1,000,000):
    - Number of printed pairs = $n^2 = 1{,}000{,}000^2 = 1{,}000{,}000{,}000{,}000$ (one trillion pairs).
- That is an **enormous** number of operations:
    - Even if each print took only a tiny fraction of a second, printing one trillion lines would take an impractically long time (many days or more), and produce a massive output.
- So, while the formal Big-O is $O(n^2)$, in practice:
    - For a dictionary with 1 million keys, this function is **not feasible to run** because it tries to print one trillion pairs.